Zwischenpräsentation 24.11

Gruppe P1b

Extraktion von Infrastrukturmerkmalen an Kreuzungen aus Street-Level-Bildern

Gliederung

1. Business Understanding
2. Data Understanding
3. Data Preparation
4. Modeling

Data Understanding

Business understanding

- **Motivation:** Die automatische Extraktion von Straßenmerkmalen (Fahrstreifen, Zebrastreifen) ist entscheidend für intelligente Städte und autonomes Fahren.

- **Derzeitige Lücke:** Obwohl Datensätze wie Mapillary existieren, fehlt eine systematische Filterung von Kreuzungen und quantitative Kennzahlen.

- **Ziel:** Entwicklung einer Deep-Learning-Pipeline (z. B. DeepLab, U-Net) zur Segmentierung und Quantifizierung von Infrastrukturelementen aus Straßenbildern.

- **Erfolgskriterien:** Ein reproduzierbarer Workflow, der hohe mIoU-Werte (mean Intersection over Union) erreicht und zuverlässige Verkehrskennzahlen extrahiert.

City Scapes Datensatz

Klassen
- 30 Klassen gesamt
- flat: road, sidewalk, parking, ...
- human: person, rider (a person on a bycicle, motorcycle, scooter, ...)
- vehicle: car, truck, bus, motorcycle, bycicle,
- ...

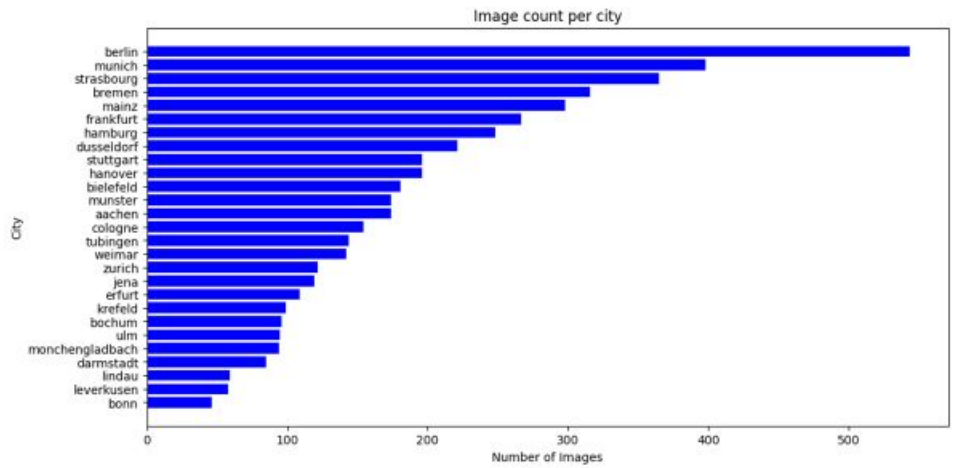
Eigenschaften
- 5,000 fine annotierte Bilder
  - von 27 Städten (überwiegend deutsche Städte)
- 20,000 coarse annotierte Bilder (können für pretraining oder data augmentation genutzt werden)
  - von 50 Städten (überwiegend deutsche Städte)

fine annotierte Bilder für das training's und validierung's set (3475 annotierte Bilder)
- 2975 für training
- 500 für validierung

dummy annotierte für das testing set (1525 schwarze Bilder)
- 1525 for val

!Jedes Bild enthält GPS daten!





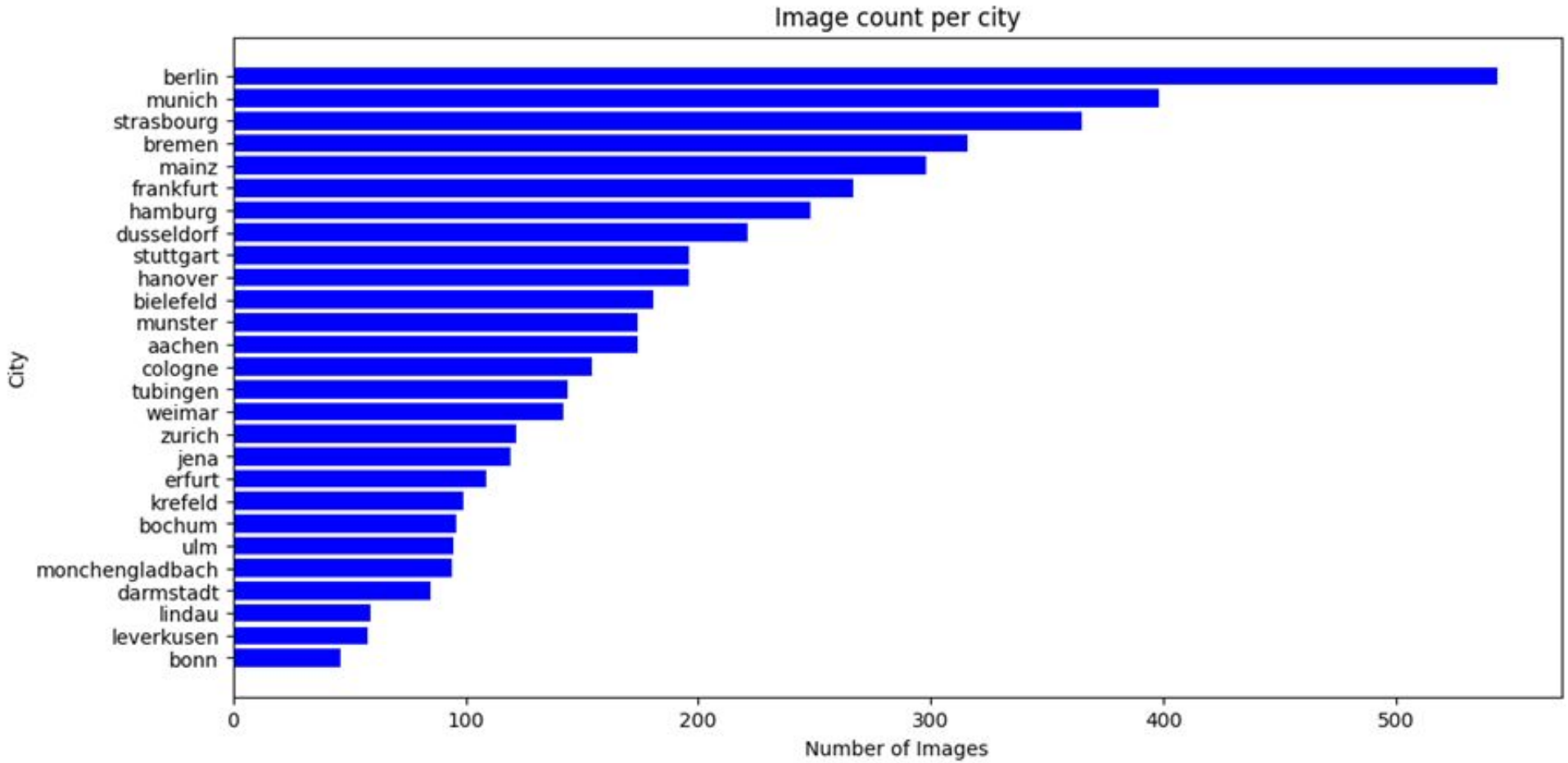

Image count per city
berlin
munich
strasbourg
bremen
mainz
frankfurt
hamburg
dusseldorf
stuttgart
hanover
bielefeld
munster
aachen
cologne
tubingen
weimar
zurich
jena
erfurt
krefeld
bochum
ulm
monchengladbach
darmstadt
lindau
leverkusen
bonn
City
0
100
200
300
400
500
Number of Images

Image GPS matching

- Für jedes Image existiert zugehörige JSON-Datei mit GPS-Daten
- Matching anhand der *image_id‘s*
  - „aachen_000000_000019_leftImg8bit.png“
  - „aachen_000000_000019_vehicle.json“
- DataFrame erstellt und als CSV exportiert
- Für *Fine* und *Coarse*

split	city	image_id	latitude	longitude
test	berlin	berlin_000000_000019	52.497751	13.295325
test	berlin	berlin_000001_000019	52.498672	13.300393
test	berlin	berlin_000002_000019	52.499890	13.306958
test	berlin	berlin_000003_000019	52.499929	13.307324
test	berlin	berlin_000004_000019	52.498154	13.306086

Mapillary Vistas Datensatz

Datensatz-Umfang & Struktur

Volumen:

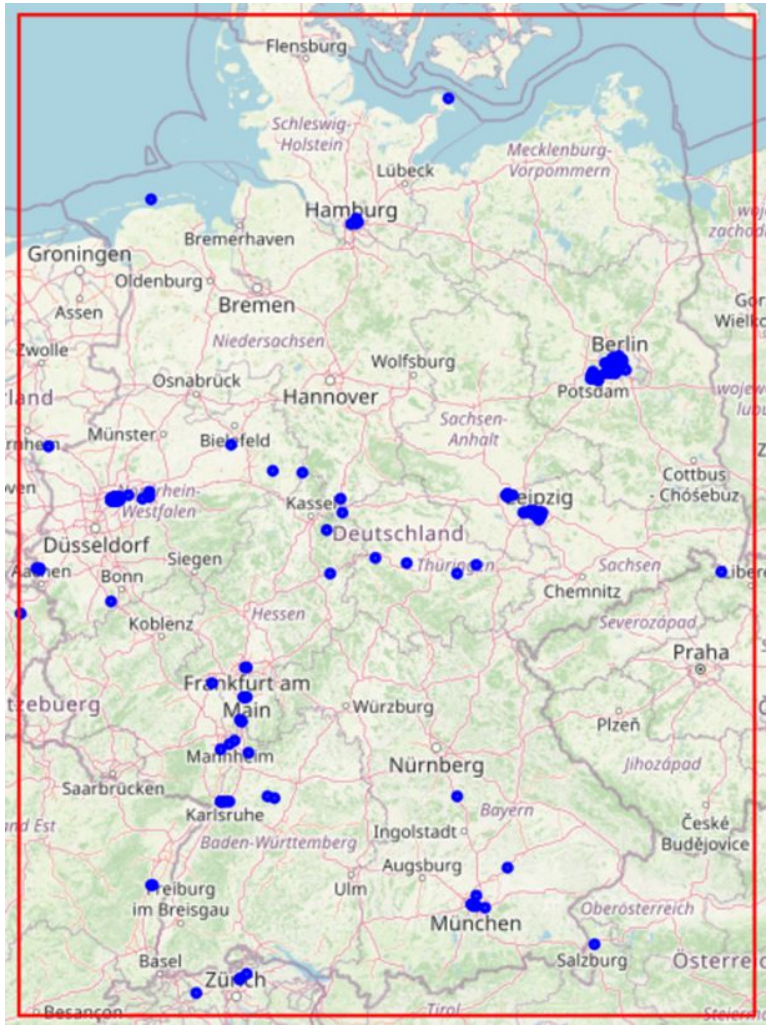
Insgesamt 25.000 hochauflösende Street-Level Bilder

Split:

18.000 Training (Training Set)
2.000 Validierung (Validation Set)
5.000 Test (Official Test Set)

Annotationstiefe & Klassen

Semantic Segmentation: 46 "Stuff"-Klassen
+ 8 "Void/Crowd"-Klassen.
Instance Segmentation: 70 Klassen



Mapillary Vistas - Semantic Segmentation



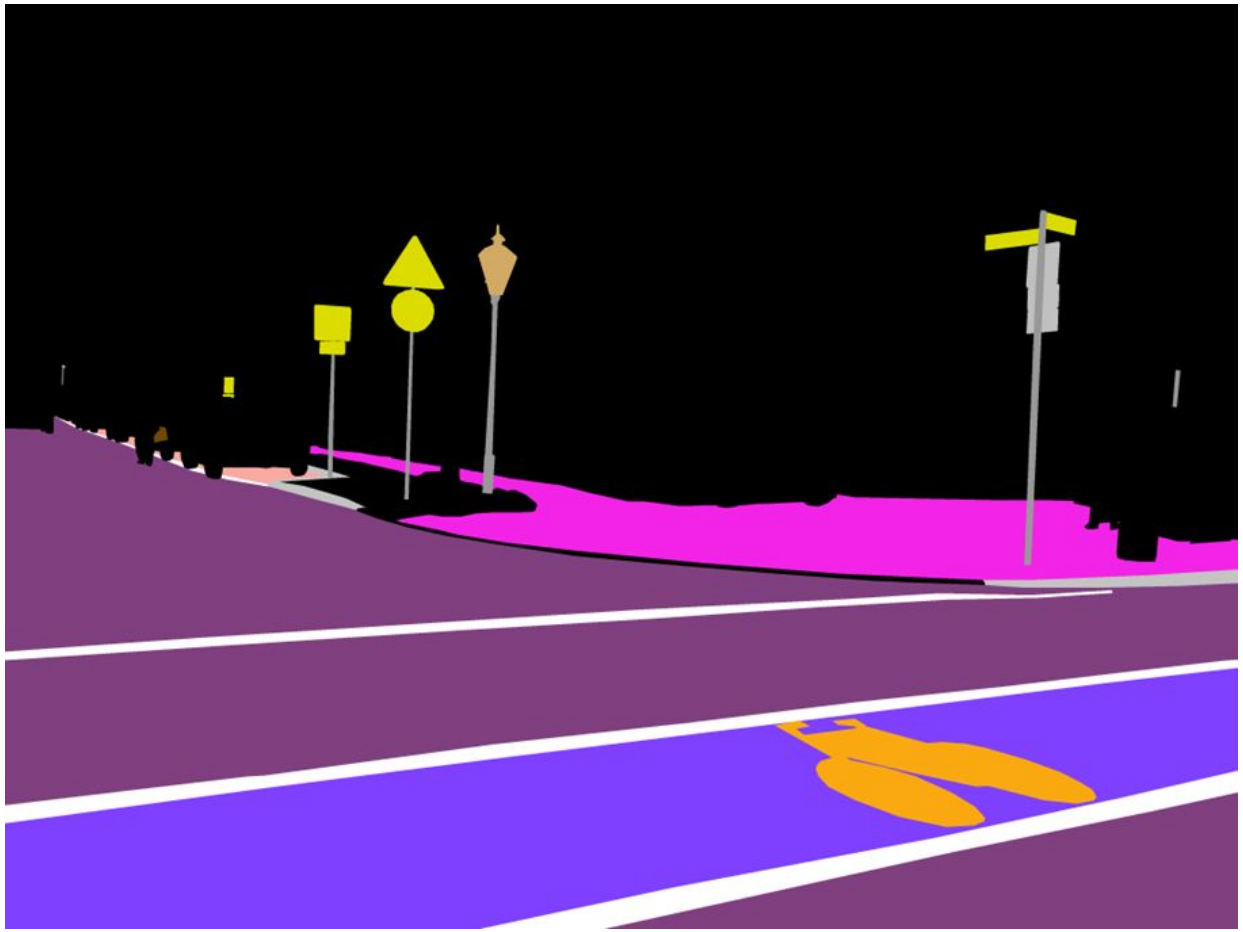


Mapillary Vistas - Semantic Segmentation



Objekt-Name	Anteil	RGB-Farbe
Vegetation	40.97%	(107, 142, 35)
Road	23.90%	(128, 64, 128)
Bike Lane	13.49%	(128, 64, 255)
Sidewalk	4.12%	(244, 35, 232)
Lane Marking (only) - Test	3.49%	(255, 255, 255)
Fence	2.31%	(190, 153, 153)
Wall	2.06%	(102, 102, 156)
Sky	1.99%	(70, 130, 180)
Lane Marking - Symbol (Bicycle)	1.37%	(250, 170, 16)
Vehicle Group	1.36%	(0, 0, 142)
Building	0.69%	(70, 70, 70)
Motorcycle	0.63%	(0, 0, 230)
Traffic Sign - Temporary (Front)	0.55%	(220, 220, 0)
Bench	0.53%	(250, 0, 30)
Static	0.50%	(111, 111, 0)
Pole Group	0.47%	(153, 153, 153)
Curb	0.33%	(196, 196, 196)
Trash Can	0.31%	(140, 140, 20)
Water Valve	0.26%	(170, 170, 170)
Person Group	0.23%	(220, 20, 60)
Traffic Sign - Temporary (Back)	0.18%	(192, 192, 192)
Street Light	0.13%	(210, 170, 100)
Parking	0.10%	(250, 170, 160)
Dynamic	0.01%	(111, 74, 0)

Mapillary Vistas - Class Mapping



Objekt-Name	Anteil	RGB-Farbe
Unlabeled	51.84%	(0, 0, 0)
Road	23.90%	(128, 64, 128)
Bike Lane	13.49%	(128, 64, 255)
Sidewalk	4.12%	(244, 35, 232)
Lane Marking (only) - Test	3.49%	(255, 255, 255)
Lane Marking - Symbol (Bicycle)	1.37%	(250, 170, 16)
Traffic Sign - Temporary (Front)	0.55%	(220, 220, 0)
Pole Group	0.47%	(153, 153, 153)
Curb	0.33%	(196, 196, 196)
Traffic Sign - Temporary (Back)	0.18%	(192, 192, 192)
Street Light	0.13%	(210, 170, 100)
Parking	0.10%	(250, 170, 160)
Dynamic	0.01%	(111, 74, 0)

➔ **Spezialisierung statt Generalisierung**

➔ **Fokussierung der Modellkapazität**

Mapillary Vistas - Semantic Segmentation



- ➔ **16-Bit Bild**
- ➔ Jedes Objekt erhält eine ID
- ➔ Kein „Verschmelzen“ von Objekten
- ➔ Ermöglicht statistische Erhebungen

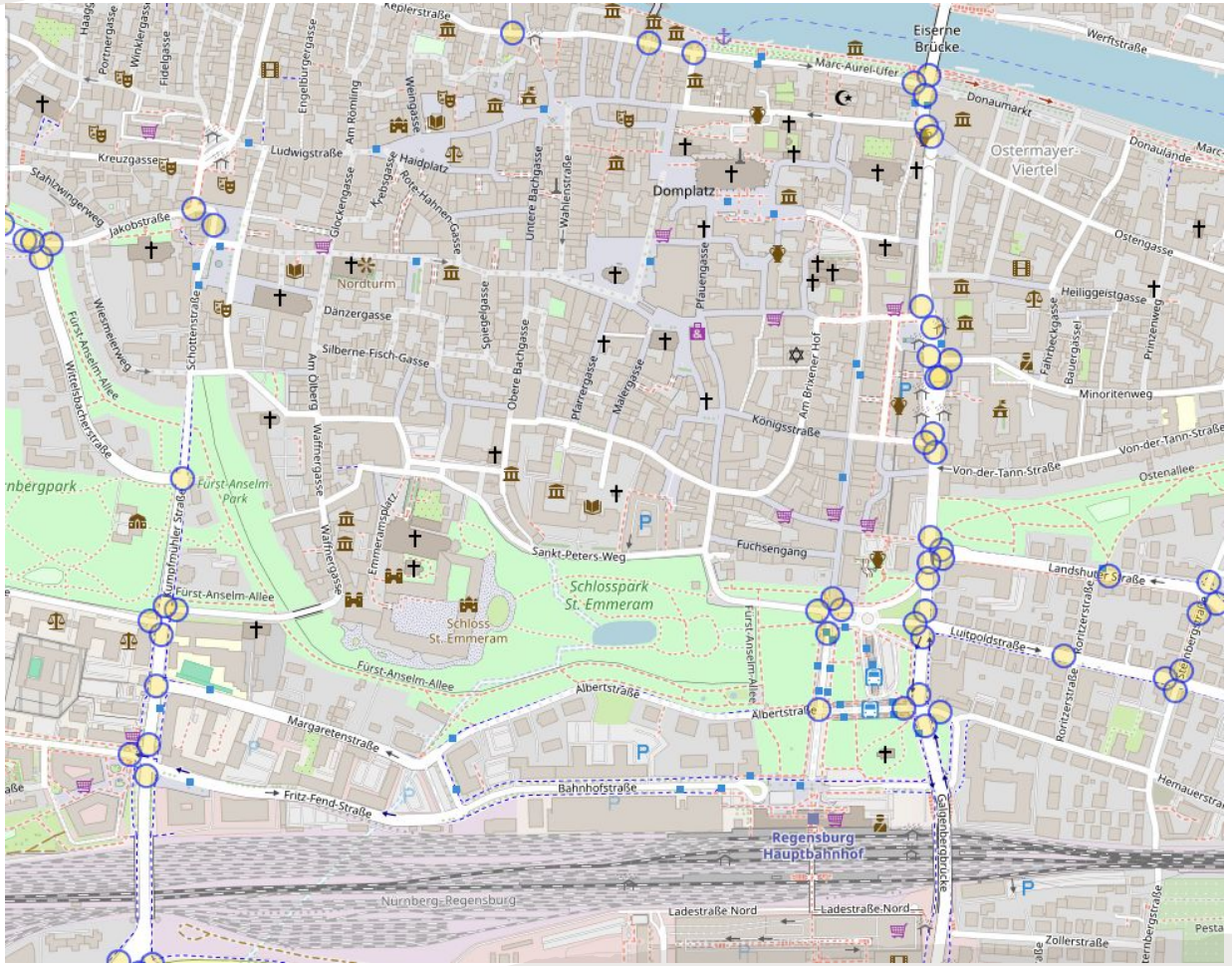
Data Preparation

Kreuzungserkennungs-Pipeline

Native Overpass API Suche

Native Overpass API Suche Methode:
Suche nach typischen Kreuzungsmerkmalen (Ampeln, Verkehrskreiseln) in Regensburg

Problem:
Diese Methode liefert keine richtigen Kreuzungen zurück. Ampeln und Verkehrskreisel sind nur Näherungswerte für echte Straßenkreuzungen.

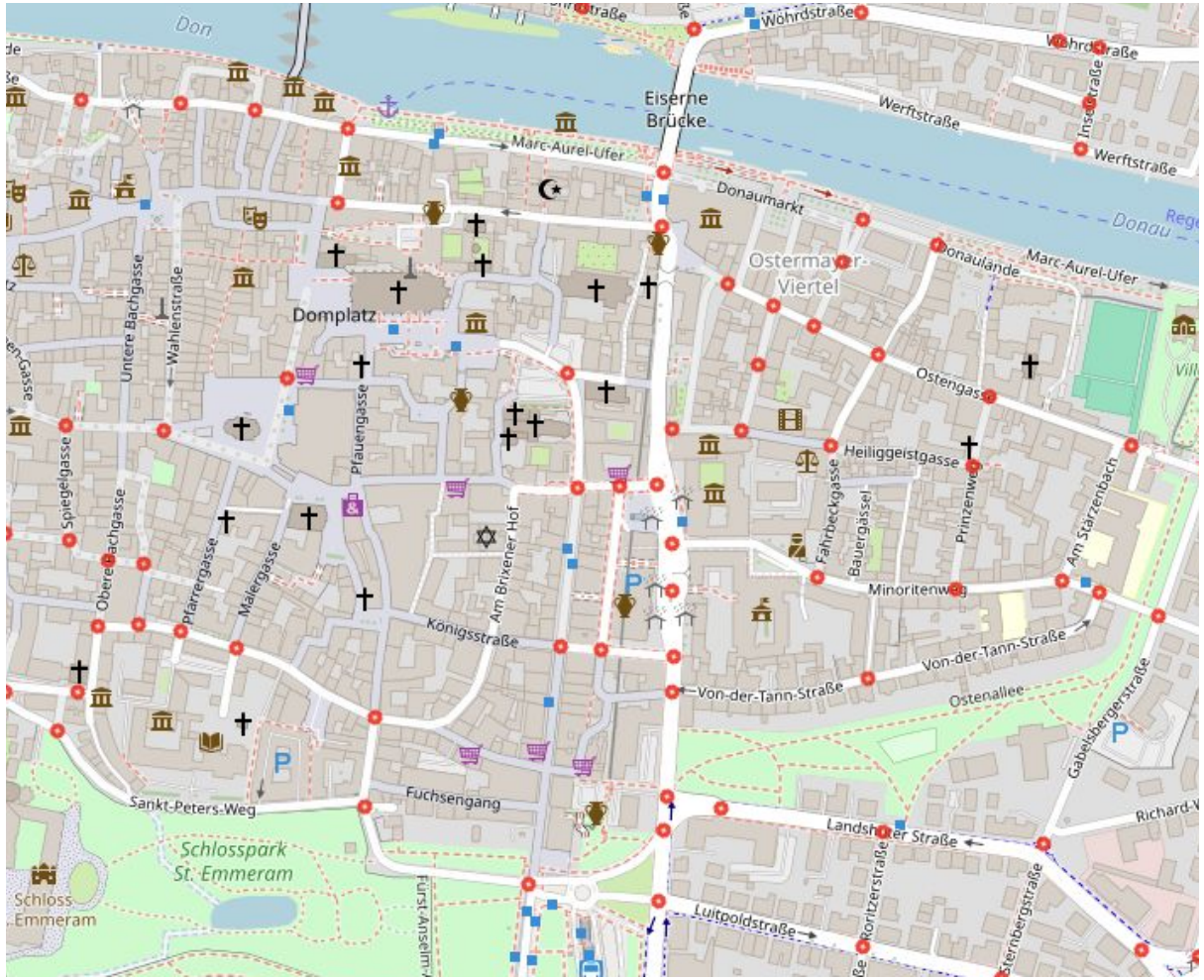


Kreuzungserkennungs-Pipeline

Lösung: Verwendung der consolidate_intersections() Funktion, die echte Kreuzungen aus Verkehrsnetzen extrahiert.

Problem bei einfacher Graphenrepräsentation: Reale, komplexe Kreuzungen (wie Kreisverkehre oder Straßen mit Mittelstreifen) werden in Graphenmodellen oft durch mehrere Knoten dargestellt, was zu einer systematischen Überzählung von Kreuzungen führt.

Lösungsansatz: Knoten werden basierend auf einem Puffer-Radius (Toleranz) räumlich zusammengefasst.

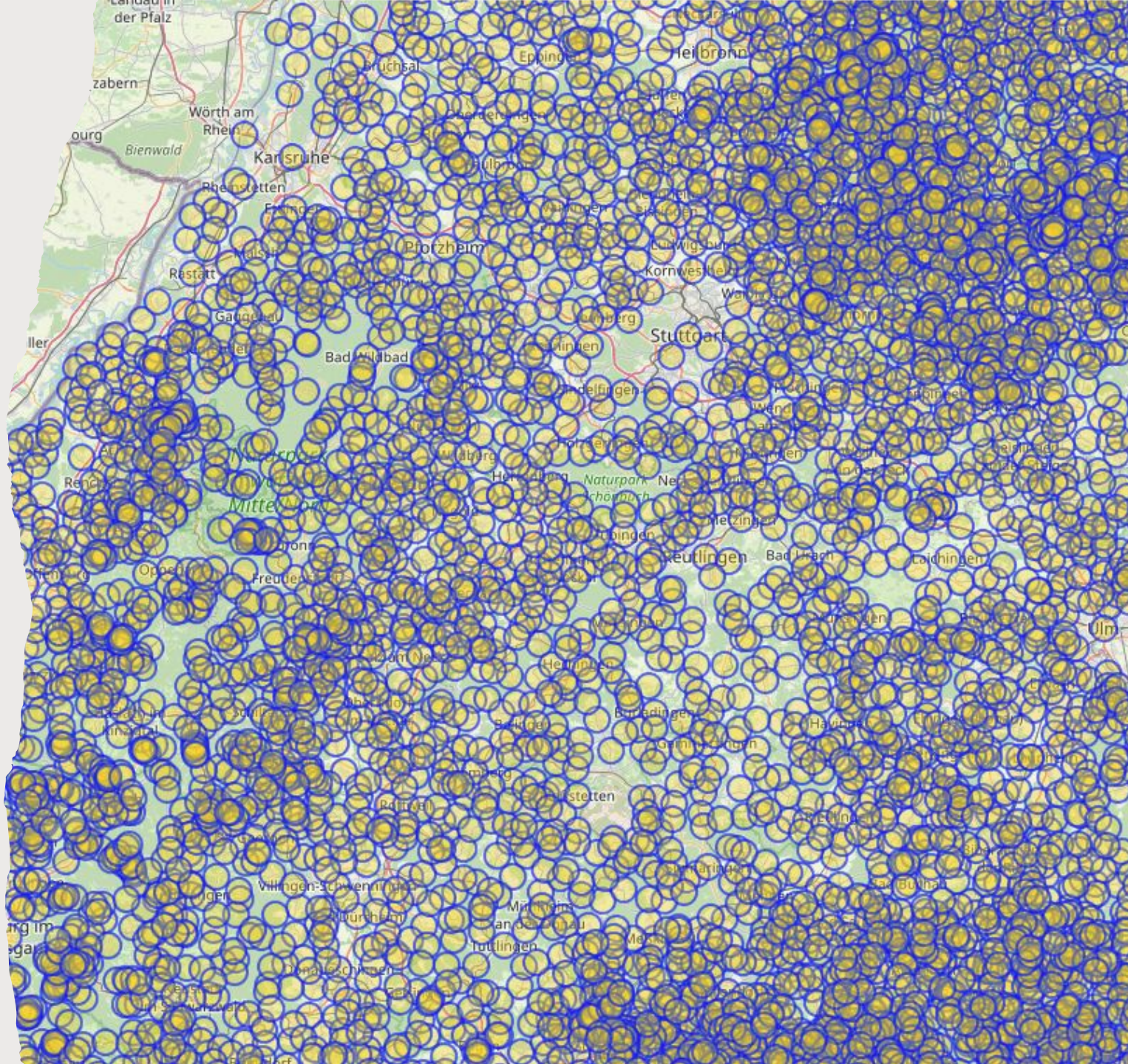


Kreuzungserkennungs-Pipeline

Deutschland gesamt Zu groß, selbst mit Batch-Aufteilung und Reduzierung von admin_level=6 (Landkreise und kreisfreie Städte) auf admin_level=8 (Städte und Gemeinden) Beispiel: Allein Städte und Gemeinden in Baden-Württemberg

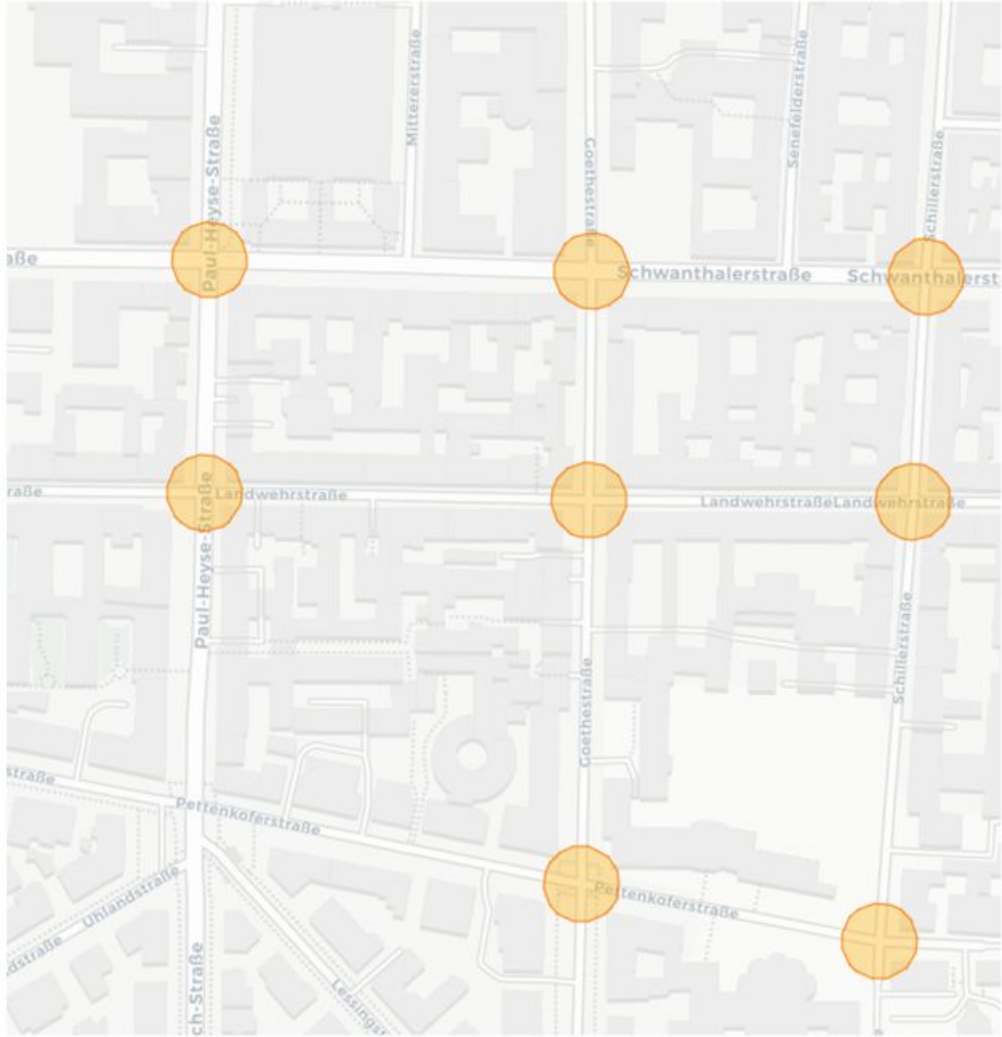
Finale Lösung Strategie: Orientierung an den in CityScapes verfügbaren Städten und Eingrenzung der Bereiche anhand dieser vorhandenen Städte, um gezielt nach Kreuzungen zu suchen.

Vorteil: Fokus auf relevante Städte mit bereits vorhandenen Bildaten, keine unnötige Verarbeitung von Regionen ohne CityScapes-Abdeckung.



Selektion relevanter Kreuzungsbilder

- **Input:** GPS-Daten aller Bilder + Koordinaten der Kreuzungen. Mit 3-Wege- und 4-Wege-Kreuzungen.
  Für Fine- und Coarse-GPS-Bilder
- Ausgangslage: einfache Koordinaten der Kreuzungen nicht ausreichend
- Lösung:Erstellung eines **30-Meter-Radius (Buffer)** um jeden Kreuzungspunkt
- **Matching (Spatial Join):** Automatische Prüfung, ob ein Bild-GPS-Punkt innerhalb dieses 30m-Kreises liegt.
- ->Konvertierung der Koordinaten mit geopandas zu einem 30 Meter Kreis



Selektion relevanter Kreuzungsbilder

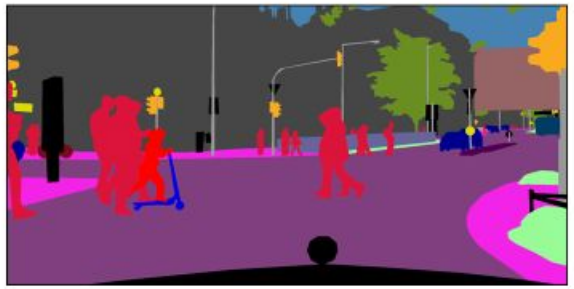
- Matching mit geopds relativ einfach
- Ergebnis
- Fine_3 images innerhalb der Puffer: 1,655
- Coarse_3 images innerhalb der Puffer: 6,269
- Fine_4 images innerhalb der Puffer: 576
- Coarse_4 images innerhalb der Puffer: 1,912

```
Unnamed: 0,split,city,image_id,latitude,longitude,intersection_id
8,test,berlin,berlin_000008_000019,52.49548361000748,13.308616285862556,355
19,test,berlin,berlin_000019_000019,52.50112857033875,13.31698653819213,1921
21,test,berlin,berlin_000021_000019,52.50151807737928,13.319364065027733,1869
31,test,berlin,berlin_000031_000019,52.50550828238575,13.325725426623356,1855
45,test,berlin,berlin_000045_000019,52.51096858939007,13.3235406835608,1852
70,test,berlin,berlin_000070_000019,52.52200805878573,13.376234607517851,545
71,test,berlin,berlin_000071_000019,52.52222330605927,13.377510752276248,548
81,test,berlin,berlin_000081_000019,52.51822857482509,13.38039957520551,563
84,test,berlin,berlin_000084_000019,52.517073297110485,13.3857043140869,628
85,test,berlin,berlin_000085_000019,52.51699609978223,13.385733231652845,628
87,test,berlin,berlin_000087_000019,52.51537579407948,13.386058449496064,890
88,test,berlin,berlin_000088_000019,52.51457968091592,13.38625735205839,897
95,test,berlin,berlin_000095_000019,52.514727202411805,13.391113742938968,921
98,test,berlin,berlin_000098_000019,52.51523942051482,13.396779909485245,968
```

Filter intersection images

- Filtern der Images und Annotations auf Kreuzungen anhand der *images_within_intersections.csv's*
- Mit der *image_id* zugehöriges Image, farbige Labelmaske und Label-Id-Maske aus dem ursprünglichen Datensatz selektieren
- Für *Fine* und *Coarse* sowie *degree = 3* und *degree = 4* der Kreuzungen filtern

split	city	image_id	latitude	longitude	intersection_id
test	berlin	berlin_000001_000019	52.498672	13.300393	1514
test	berlin	berlin_000002_000019	52.499890	13.306958	1783
test	berlin	berlin_000005_000019	52.496816	13.305182	2742
test	berlin	berlin_000006_000019	52.496830	13.307894	2745
test	berlin	berlin_000007_000019	52.493911	13.310314	2424





degree = 4









degree = 4









degree = 3





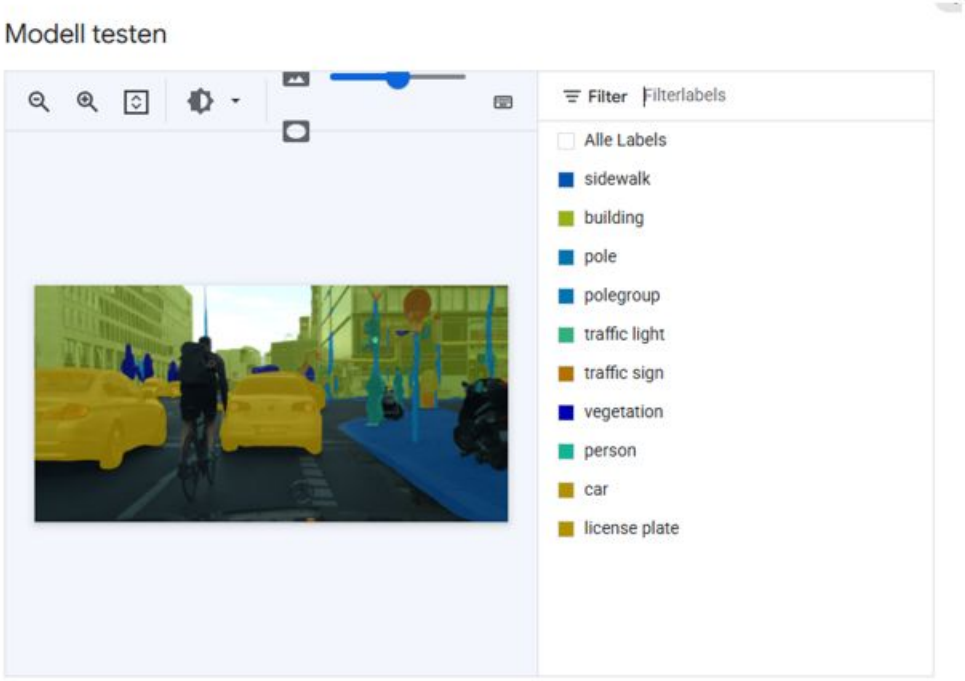




Modeling

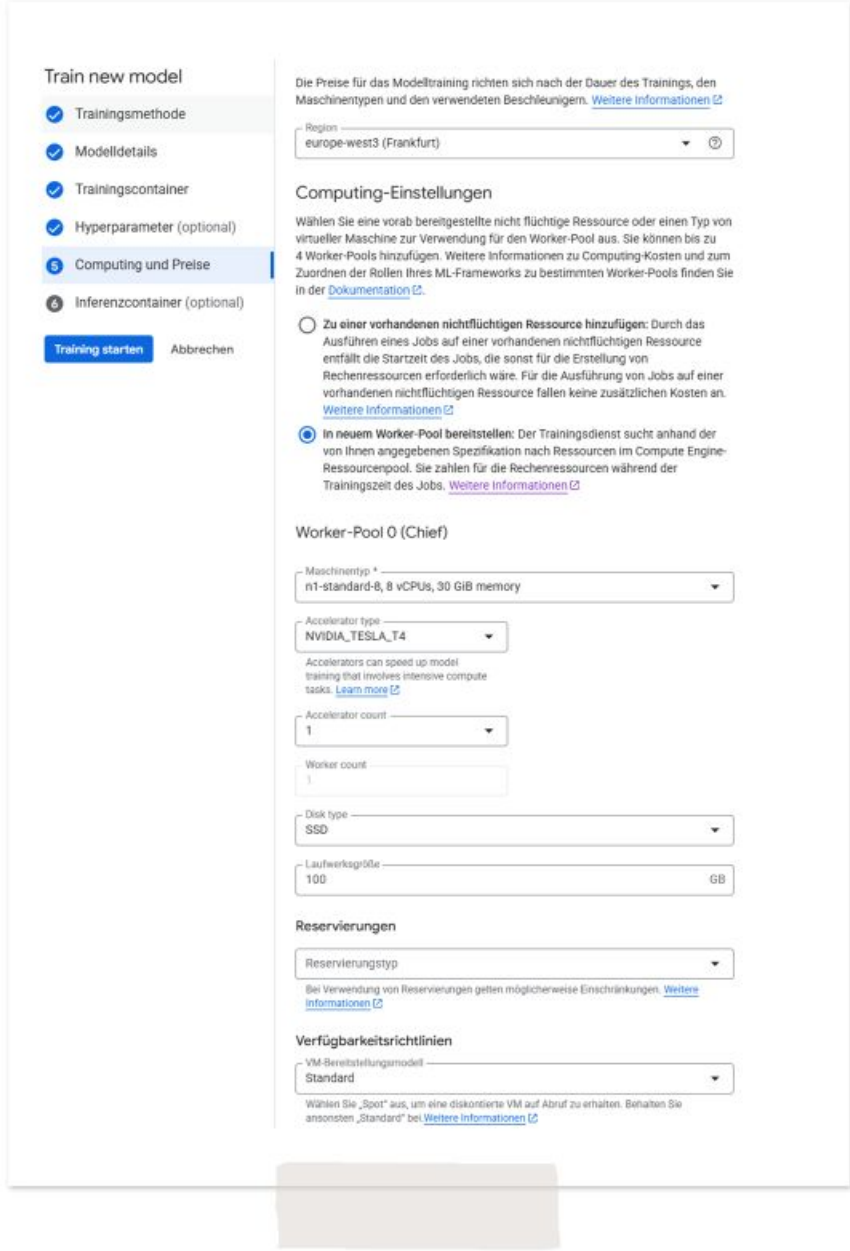
DeepLabv3+ (with checkpoint)

- Google Cloud Console (Vertex AI)
- Bietet ein Vortrainiertes Model
  -> Trainiert mit cityscapes
- Vorteil: Starke Zeitersparnis



DeepLabv3+ (with checkpoint)

- Vorgehen :
    1. Speichern der gefilterten Bilder im Cloud Storage(Bucket)
    2. Benutzerdefinierten Job in Vertex Ai unter „training“ erstellen
    3. Neues Modell erstellen
    4. Pre-built TensorFlow Container nutzen
    5. Optional (Hyperparameter tuning) -> erhöhte Kosten
    6. Computing Infrastruktur wählen
    7. Trainieren



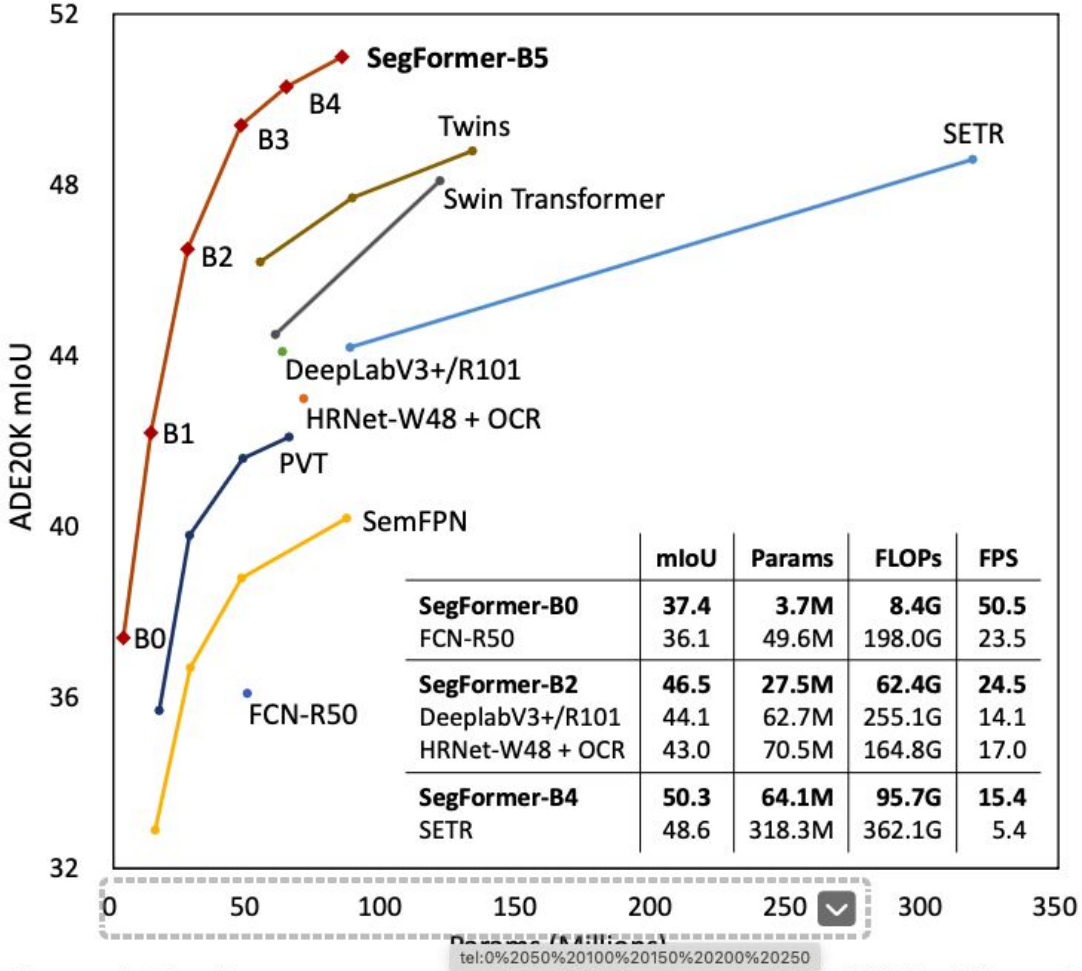
Vortrainierte Modelle

SegFormer ist ein Framework für die semantische Segmentierung.

Es wurde auf mehreren Datensätzen trainiert

Transformer-Encoder (MiT) zur multiskaligen Merkmalsextraktion

Das Risiko: Eine Evaluierung auf Cityscape Bildern ist sinnlos. Das Modell kennt die Texturen, Lichtverhältnisse und Kamerawinkel bereits.



	mIoU	Params	FLOPs	FPS
SegFormer-B0	**37.4**	**3.7M**	**8.4G**	**50.5**
FCN-R50	36.1	49.6M	198.0G	23.5
SegFormer-B2	**46.5**	**27.5M**	**62.4G**	**24.5**
DeeplabV3+/R101	44.1	62.7M	255.1G	14.1
HRNet-W48 + OCR	43.0	70.5M	164.8G	17.0
SegFormer-B4	**50.3**	**64.1M**	**95.7G**	**15.4**
SETR	48.6	318.3M	362.1G	5.4

Figure 1: **Performance *vs.* model efficiency on ADE20K.** All results are reported with single model and single-scale inference. SegFormer achieves a new state-of-the-art 51.0% mIoU while being significantly more efficient than previous methods.

Nächste Schritte

- SegFormer-Modell auf neuen Daten testen und validieren
- Weitere geeignete Modelle identifizieren
- Trainieren eines eigenen Segmentierungsmodells